Pink Notes

A pink-themed note-taking web app built with **Flask** and **SQLite**, deployable as a single file with no external dependencies beyond Flask.

Features

- **Create, read, update, delete** notes with title and content
- **Reset database** — wipe all notes with one click
- **Backup** — download entire database as a `.sql` file
- **Restore** — upload a `.sql` backup to replace all data
- **Animations** — fade-in, slide-in note cards, sparkle burst on save, pulsing hearts
- **Pink girlish UI** — hot pink headers, soft pink accents, rounded cards

---

Project Structure

```
cloud/
 server.py          # single-file Flask app (HTML inlined)
 render.yaml        # Render Blueprint for one-click deploy
 templates/
   index.html       # standalone HTML (for reference; not used at runtime)
 requirements.txt   # flask>=3.0
 .gitignore         # ignores notes.db, backup.sql, __pycache__
```

> **Note:** `server.py` serves the HTML directly from an inline string — the `templates/` folder exists only for reference. Deployment requires only `server.py`.

Code Architecture

Backend: `server.py`

The app is a single Flask file organized into three layers:

1. Database Layer

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_PATH = os.path.join(BASE_DIR, "notes.db")

def get_db():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    conn.execute("""
        CREATE TABLE IF NOT EXISTS notes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL,
            content TEXT NOT NULL DEFAULT '',
            created_at TEXT NOT NULL,
            updated_at TEXT NOT NULL
        )
    """)
    conn.commit()
    return conn
```

- Uses `sqlite3` from Python stdlib — no extra install
- `notes.db` is created automatically next to `server.py` on first run
- `row_factory = sqlite3.Row` allows accessing columns by name (`row["title"]`)

2. API Routes (REST)

Route	Method	Description
`/`	GET	Serves the SPA (single-page app)
`/api/notes`	GET	List all notes (newest first)
`/api/notes`	POST	Create a note `{title, content}`
`/api/notes/<id>`	PUT	Update a note
`/api/notes/<id>`	DELETE	Delete a single note
`/api/backup`	GET	Download database as `.sql` file
`/api/restore`	POST	Upload `.sql` file, replace database
`/api/reset`	POST	Drop and recreate the notes table

3. Frontend (inlined HTML/JS)

The entire HTML is stored as a Python raw string `HTML = r'''...'''` and served via:

```
@app.route("/")
def index():
    return HTML
```

This eliminates the need for Jinja templates, making deployment a single-file operation.

Frontend: SPA (Single Page Application)

The HTML/CSS/JS implements a full client-side app:

- **State management:** `currentId` tracks whether editing an existing note or creating a new one
- **API calls:** All `fetch()` requests hit the `/api/` endpoints
- **Animations:** Pure CSS keyframes (`fadeIn`, `slideIn`, `sparkle`, `pulse`)
- **Sparkle effect:** JavaScript spawns floating emoji particles on save
- **Toast notifications:** Temporary status messages

CSS Variables (Pink Theme)

```
:root {
  --bg: #FFF0F5;       /* lavender blush background */
  --card: #FFFFFF;     /* note card background */
  --accent: #FF69B4;   /* hot pink (header, borders, save button) */
  --dark: #FF1493;     /* deep pink (hover states) */
  --light: #FFB6C1;    /* light pink (reset button, input borders) */
  --text: #5C0040;     /* dark plum text */
  --subtle: #C488A0;   /* muted labels and dates */
}
```

Local Setup

Prerequisites

- **Python 3.10+**
- **Flask 3.0+**

Install & Run

```
# clone the repo
git clone https://github.com/Ajigiwe/Gina.git
```

```
cd Gina

# install flask
pip install -r requirements.txt

# run the app
python server.py
```

Open **http://127.0.0.1:5050** in your browser.

Deployment

PythonAnywhere (Recommended — Free)

PythonAnywhere is ideal because it has a persistent filesystem (SQLite data survives restarts), pre-installed Flask, and a free tier with no credit card required.

Step-by-Step

Sign up at pythonanywhere.com (free "Beginner" account)

Upload `server.py`

3. Dashboard → **Files** tab
4. Navigate to `/home/yourusername/`

Upload `server.py`

Create a web app

7. Dashboard → **Web** tab → **Add a new web app**
8. Choose **Flask** framework, pick **Python 3.10+**

It creates a default app — you'll replace it

Edit the WSGI file

11. On the Web tab, find the WSGI configuration file link (e.g. `/var/www/yourusername_pythonanywhere_com_wsgi.py`)
12. Replace its contents with:

```python import sys path = '/home/yourusername' if path not in sys.path: sys.path.append(path)

from server import app as application ```
```

> Replace `yourusername` with your actual PythonAnywhere username.

Reload — click the green **Reload** button on the Web tab

Visit `https://yourusername.pythonanywhere.com`

Troubleshooting

Problem	Fix
`TemplateNotFound`	You're using an old version that needs `templates/`. Upload the latest `server.py` (HTML is inlined).
`500 Internal Server Error`	Check the **Error log** on the Web tab.
Flask not found	The free tier has Flask pre-installed. If missing, open a Bash console and run `pip install --user flask`.
App not updating after upload	Hit the green **Reload** button on the Web tab.

Render (Free Tier)

Render offers a free web service tier. The app is configured via `render.yaml` for one-click deploy.

Important: Render's free filesystem is ephemeral — `notes.db` is lost on each deploy or restart. Use the **backup** button regularly to save your data, and **restore** after redeploys.

Option A: Blueprint (One-Click)

1. Push this repo to GitHub
2. Go to dashboard.render.com → **New** → **Blueprint**
3. Connect your GitHub repo — Render auto-detects `render.yaml`
4. Click **Apply**

Option B: Manual Web Service

1. Dashboard → **New** → **Web Service**
2. Connect your repo
3. Settings:
4. **Environment:** Python 3
5. **Build Command:** `pip install -r requirements.txt`

6. **Start Command:** `gunicorn server:app`
7. **Plan:** Free
8. Click **Create Web Service**

Your app will be live at `https://pink-notes.onrender.com` (or your chosen name).

Other Platforms

Platform	Notes
Fly.io	Free tier with persistent volumes. Requires credit card for verification and a `Dockerfile`.
Vercel / Netlify	Static hosting only. Not suitable for Flask + SQLite without serverless adaptation.

API Reference

List Notes

```
GET /api/notes
```

Response: `[{id, title, content, created_at, updated_at}, ...]`

Create Note

```
POST /api/notes
Content-Type: application/json
Body: {"title": "Hello", "content": "World"}
```

Response: `{id, title, content, created_at, updated_at}` (201)

Update Note

```
PUT /api/notes/<id>
Content-Type: application/json
Body: {"title": "Updated", "content": "New content"}
```

Delete Note

```
DELETE /api/notes/<id>
```

Response: `{"ok": true}`

Reset Database

```
POST /api/reset
```

Response: `{"ok": true}` — drops and recreates the notes table

Backup

```
GET /api/backup
```

Response: `.sql` file download (SQLite dump)

Restore

```
POST /api/restore
Content-Type: multipart/form-data
Body: file=<backup.sql>
```

Response: `{"ok": true}` — replaces entire database

Tech Stack

- **Backend:** Python 3, Flask, SQLite3
- **Frontend:** HTML5, CSS3 (custom properties, keyframe animations), vanilla JavaScript (fetch API)
- **Deployment:** PythonAnywhere (WSGI), any Flask-compatible host